

1. Explique as diferenças entre SCRUM e Kanban. Em que situações seria mais adequado usar cada metodologia?

Embora ambas sejam metodologias ágeis focadas em eficiência e entrega de valor, SCRUM e Kanban têm filosofias e estruturas bastante diferentes.

O **SCRUM** organiza o trabalho em ciclos fixos chamados sprints, com duração de 2 a 4 semanas. Cada sprint tem um objetivo definido, um conjunto de tarefas selecionadas previamente e uma entrega esperada ao final. A metodologia é estruturada, com papéis bem definidos (Product Owner, Scrum Master e Time de Desenvolvimento) e cerimônias obrigatórias como planejamento da sprint, daily meetings, revisão e retrospectiva.

O **Kanban** não trabalha com ciclos fixos — o trabalho flui de forma contínua, sem sprints definidos. As tarefas entram no quadro conforme surgem e avançam pelas colunas até a conclusão. Não há papéis formais obrigatórios e a principal regra é a limitação do WIP (Work in Progress), ou seja, o número máximo de tarefas que podem estar em andamento simultaneamente em cada coluna.

Quando usar cada um:

O SCRUM é mais adequado para projetos com escopo definido e entregas planejadas, onde a equipe trabalha em conjunto em funcionalidades completas dentro de um prazo. Exemplos: desenvolvimento de um novo aplicativo, criação de um sistema do zero, projetos com cliente externo que exige demonstrações periódicas.

O Kanban é mais adequado para equipes que lidam com demandas contínuas e imprevisíveis, onde novas tarefas surgem o tempo todo e não é possível planejar sprints com antecedência. Exemplos: equipes de suporte e manutenção de sistemas, times de operações de TI, correção de bugs em sistemas em produção.

2. Descreva as responsabilidades de um Product Owner no SCRUM. Por que esse papel é importante?

O **Product Owner (PO)** é o responsável por representar os interesses do cliente e do negócio dentro do time SCRUM. Ele é o elo entre quem paga pelo produto e quem o constrói, garantindo que a equipe de desenvolvimento sempre esteja trabalhando no que gera mais valor.

Responsabilidades principais:

Gestão do Product Backlog: O PO cria, mantém e prioriza a lista de todas as funcionalidades, melhorias e correções que o produto precisa. Ele decide o que entra no backlog, o que é removido e, principalmente, qual é a ordem de prioridade.

Definição de critérios de aceitação: Para cada item do backlog, o PO define claramente quando uma funcionalidade pode ser considerada "pronta" — quais são os requisitos mínimos que precisam ser atendidos para que a entrega seja aceita.

Participação no planejamento da sprint: No início de cada sprint, o PO apresenta e explica os itens do backlog para a equipe, esclarece dúvidas e negocia quais funcionalidades serão desenvolvidas no ciclo.

Validação das entregas: Ao final de cada sprint, o PO avalia se o que foi desenvolvido atende às expectativas e decide se a funcionalidade está aprovada para entrar em produção.

Por que é importante: Sem o Product Owner, o time de desenvolvimento correria o risco de construir funcionalidades tecnicamente corretas, mas que não atendem à necessidade real do cliente ou do negócio. O PO garante que cada hora de trabalho da equipe seja direcionada ao que realmente importa, evitando desperdício de recursos e retrabalho. Ele é, essencialmente, o guardião do valor do produto.

3. Explique como a programação em par do XP pode melhorar a qualidade do código. Dê um exemplo.

A **programação em par (pair programming)** é uma prática do Extreme Programming em que dois desenvolvedores trabalham juntos em uma única estação de trabalho — um escreve o código (driver) enquanto o outro revisa em tempo real, sugere melhorias e identifica possíveis problemas (navigator). Os papéis se alternam regularmente.

Como melhora a qualidade do código:

Revisão em tempo real: Erros de lógica, bugs e falhas de segurança são identificados no momento em que o código é escrito, antes de qualquer teste formal. Isso é muito mais eficiente do que uma revisão posterior, quando o erro já pode ter se propagado para outras partes do sistema.

Disseminação de conhecimento: O conhecimento sobre o código nunca fica concentrado em uma única pessoa. Ambos os desenvolvedores entendem o que foi escrito, o que facilita manutenção futura e elimina o risco do chamado "ônibus factor" — quando apenas uma pessoa conhece determinada parte crítica do sistema.

Código mais limpo e bem estruturado: Saber que outro desenvolvedor está observando em tempo real naturalmente incentiva a escrever código mais claro, com nomes de variáveis mais descritivos e melhor organização, seguindo boas práticas que sozinho o desenvolvedor poderia negligenciar por pressa.

Exemplo prático: Uma equipe está desenvolvendo o módulo de autenticação de uma API — uma parte crítica do sistema que lida com senhas e tokens de segurança. Com a programação em par, enquanto um desenvolve a lógica de geração do token JWT, o outro verifica em tempo real se os padrões de segurança estão sendo seguidos (hash de senha, tempo de expiração do token, validação de entrada). Esse tipo de verificação simultânea em código sensível reduz drasticamente o risco de vulnerabilidades de segurança passarem despercebidas.

4. Liste e explique as fases do ciclo de vida de um projeto de software. Dê um exemplo prático para cada fase.

Utilizarei como contexto um projeto de **sistema de agendamento para clínicas médicas**:

Fase 1 — Planejamento: Define os objetivos do projeto, o escopo, o cronograma, o orçamento e os recursos necessários. É quando se decide o que será feito, por quem, em quanto tempo e com qual custo. *Exemplo:* A equipe define que o sistema terá módulos de agendamento, cadastro de pacientes e prontuário eletrônico, com prazo de 4 meses e orçamento de R\$ 80.000.

Fase 2 — Análise de Requisitos: Levantamento detalhado das necessidades do cliente, transformadas em especificações técnicas que guiarão o desenvolvimento. *Exemplo:* Entrevistas com médicos e recepcionistas revelam que o sistema precisa enviar lembretes automáticos de consulta por WhatsApp e que prontuários devem ser acessíveis offline em tablets.

Fase 3 — Design: Projeto da arquitetura do sistema, das interfaces de usuário e da forma como os componentes se comunicam. *Exemplo:* A equipe cria wireframes das telas no Figma, define o banco de dados relacional com as tabelas de pacientes, médicos e consultas, e decide que o sistema usará arquitetura cliente-servidor com API REST.

Fase 4 — Desenvolvimento: Escrita do código, implementação das funcionalidades e integração dos componentes do sistema. *Exemplo:* Os desenvolvedores constroem o módulo de agendamento, implementam a integração com a API do WhatsApp para envio de lembretes e desenvolvem o sistema de login com diferentes perfis de acesso.

Fase 5 — Testes: Verificação de que o software funciona conforme especificado, identificação e correção de falhas. *Exemplo:* A equipe realiza testes de unidade em cada módulo, testes de integração entre agendamento e envio de lembretes, e testes de aceitação com usuários reais da clínica para validar a usabilidade.

Fase 6 — Entrega e Implantação: Instalação do sistema no ambiente de produção e disponibilização para os usuários finais. *Exemplo:* O sistema é implantado no servidor da clínica, os dados dos pacientes existentes são migrados e a equipe realiza treinamento com médicos e recepcionistas.

Fase 7 — Manutenção: Suporte contínuo, correção de bugs identificados em uso real e implementação de melhorias. *Exemplo:* Um mês após o lançamento, os usuários reportam que o lembrete de WhatsApp não está sendo enviado para números internacionais. A equipe identifica e corrige o bug na validação do formato do telefone.

5. Crie um quadro Kanban para um projeto de desenvolvimento de aplicativo móvel.

A FAZER	EM PROGRESSO	CONCLUÍDO
Criar tela de perfil do usuário	Desenvolver sistema de notificações push	Definição de requisitos do app
Implementar filtro de busca avançada	Integração com API de pagamento	Criação dos wireframes no Figma
Desenvolver módulo de histórico de pedidos		Configuração do ambiente de desenvolvimento
Criar testes automatizados para o módulo de login		
Implementar modo escuro (dark mode)		

6. Descreva como a integração contínua pode ser usada para melhorar o fluxo de trabalho em um projeto ágil.

A **Integração Contínua (CI — Continuous Integration)** é a prática de integrar o código de todos os desenvolvedores em um repositório compartilhado com alta frequência — idealmente várias vezes ao dia —, com cada integração sendo verificada automaticamente por um conjunto de testes.

Como melhora o fluxo de trabalho:

Em projetos sem CI, é comum que desenvolvedores trabalhem em suas branches isoladas por dias ou semanas, e quando tentam integrar o código de todos ao mesmo tempo, surgem conflitos massivos e bugs difíceis de rastrear — um problema conhecido como "integration hell". A CI elimina esse problema ao integrar continuamente, mantendo os conflitos pequenos e fáceis de resolver.

Cada vez que um desenvolvedor faz um push no repositório, o pipeline de CI dispara automaticamente: compila o código, executa todos os testes automatizados e reporta se algo quebrou. Se um erro for detectado, a equipe é notificada imediatamente e o problema pode ser corrigido enquanto o contexto ainda está fresco na mente do desenvolvedor.

Exemplo prático: Uma equipe de 5 desenvolvedores trabalhando em um sistema de e-commerce usa o GitHub Actions como ferramenta de CI. Cada vez que qualquer desenvolvedor abre um Pull Request, o pipeline automaticamente executa 300 testes unitários e de integração. Se todos passam, o código pode ser revisado e aprovado para merge. Se algum teste falha, o PR fica bloqueado até a correção. Isso garante que a branch principal do projeto nunca tenha código quebrado, e que todas as integrações de novos módulos — como um novo método de pagamento — não comprometam funcionalidades já existentes.

7. Explique como os testes automatizados contribuem para o controle de qualidade no desenvolvimento de software.

Os testes automatizados são scripts de código que verificam automaticamente se o software se comporta conforme esperado, sem necessidade de intervenção humana manual a cada verificação. Eles são executados de forma rápida e repetível a qualquer momento, especialmente a cada nova alteração no código.

Contribuições para o controle de qualidade:

Deteção precoce de regressões: Quando um desenvolvedor implementa uma nova funcionalidade, os testes automatizados verificam instantaneamente se algo que funcionava antes parou de funcionar. Isso é chamado de teste de regressão, e sem automação seria inviável realizar essa verificação manualmente em todas as funcionalidades do sistema a cada alteração.

Redução do custo de correção: Estudos na área de engenharia de software mostram consistentemente que o custo de corrigir um bug aumenta exponencialmente conforme ele avança no ciclo de desenvolvimento. Um bug encontrado nos testes automatizados durante o desenvolvimento custa muito menos do que o mesmo bug encontrado em produção pelo cliente.

Documentação viva do comportamento esperado: Testes bem escritos descrevem exatamente o que o sistema deve fazer em cada situação. Um novo desenvolvedor que entra no projeto pode ler os testes para entender como cada funcionalidade deve se comportar — é uma forma de documentação que nunca fica desatualizada, porque se o código mudar e o teste não for atualizado, ele simplesmente falha.

Confiança para refatorar: Sem testes, a equipe evita refatorar (melhorar a estrutura do código) por medo de quebrar algo. Com uma suite de testes robusta, a equipe pode fazer melhorias arquiteturais com segurança, sabendo que os testes vão alertar imediatamente se algo quebrar.

Tipos principais: Testes de unidade (verificam funções isoladas), testes de integração (verificam a comunicação entre módulos) e testes end-to-end (simulam o comportamento do usuário no sistema completo) formam uma pirâmide de cobertura que garante qualidade em diferentes níveis do sistema.

8. Simule a organização de uma sprint no SCRUM para o desenvolvimento de um módulo de login.

Sprint 1 — Módulo de Login e Autenticação

Duração: 2 semanas (30 de abril a 13 de maio de 2026)

Objetivo da Sprint: Entregar um sistema de autenticação funcional e seguro, permitindo que usuários se cadastrem, façam login com e-mail e senha, e recuperem a senha em caso de esquecimento.

Critério de aceitação: Um usuário consegue criar uma conta, fazer login, ser redirecionado ao dashboard e recuperar o acesso via e-mail, sem erros e com tempo de resposta abaixo de 2 segundos.

Product Backlog selecionado para a Sprint:

#	Tarefa	Responsável	Estimativa
1	Criar tela de cadastro com validação de campos (nome, e-mail, senha)	Dev Front-end	6h
2	Desenvolver endpoint de registro de usuário na API (POST /auth/register)	Dev Back-end	5h
3	Implementar criptografia de senha com bcrypt	Dev Back-end	3h
4	Criar tela de login com campos de e-mail e senha	Dev Front-end	4h
5	Desenvolver endpoint de autenticação e geração de token JWT	Dev Back-end	6h
6	Implementar validação e expiração do token JWT no front-end	Dev Front-end	5h
7	Criar fluxo de recuperação de senha via e-mail (link com token temporário)	Dev Back-end	8h
8	Criar tela de redefinição de senha	Dev Front-end	4h
9	Escrever testes automatizados para todos os endpoints de autenticação	Dev Back-end	6h
10	Testes de aceitação e correções finais	Toda a equipe	5h

Total estimado: 52 horas

Cerimônias da Sprint:

- **Sprint Planning:** 30/04 — Definição do objetivo e seleção das tarefas
- **Daily Scrum:** Todos os dias úteis, 15 minutos pela manhã
- **Sprint Review:** 13/05 — Demonstração do módulo de login funcionando

- **Retrospectiva:** 13/05 (após a Review) — O que funcionou bem? O que melhorar?
-

9. Pesquise uma ferramenta de integração contínua e descreva como ela pode ser implementada em um projeto de software.

Ferramenta: GitHub Actions

O GitHub Actions é uma plataforma de automação integrada diretamente ao GitHub, que permite criar pipelines de CI/CD por meio de arquivos de configuração no formato YAML armazenados no próprio repositório do projeto. Por estar integrado ao GitHub, não há necessidade de configurar servidores externos ou ferramentas separadas — o pipeline é definido junto ao código.

Como funciona:

O desenvolvedor cria um arquivo de configuração (workflow) dentro da pasta `.github/workflows/` do repositório. Esse arquivo define os gatilhos (quando o pipeline será executado — por exemplo, a cada push ou a cada Pull Request na branch main) e os jobs (conjunto de etapas a serem executadas).

Exemplo de implementação em um projeto Node.js:

O pipeline seria configurado para, a cada Pull Request aberto, executar automaticamente as seguintes etapas: fazer checkout do código, instalar as dependências do projeto com npm install, executar o linter para verificar erros de formatação e padrão de código, rodar todos os testes automatizados com npm test, e gerar um relatório de cobertura de código. Se qualquer etapa falhar, o Pull Request fica marcado com erro e não pode ser aprovado para merge até a correção.

Benefícios práticos da implementação:

O GitHub Actions é gratuito para repositórios públicos e oferece um generoso limite de horas gratuitas para repositórios privados, o que o torna ideal para startups e projetos acadêmicos. Por estar no mesmo ambiente do código, novos membros da equipe não precisam configurar nada externamente — o pipeline já está lá quando clonam o repositório. Isso reduz atrito na adoção e garante que todos estejam usando o mesmo processo de verificação de qualidade.

10. Explique como o ciclo de vida de um projeto de software pode ser adaptado ao uso de metodologias ágeis.

O ciclo de vida tradicional de software, conhecido como modelo Cascata (Waterfall), executa as fases de forma sequencial e linear — planejamento completo, depois análise total de requisitos, depois design inteiro, depois desenvolvimento, depois testes e finalmente entrega. O problema desse modelo é que o cliente só vê o produto funcionando ao final, quando mudanças são caríssimas e demoradas.

As metodologias ágeis como SCRUM e XP adaptam esse ciclo de forma iterativa e incremental — em vez de executar cada fase uma única vez de forma completa, o projeto percorre versões comprimidas de todas as fases em cada sprint ou iteração.

Como fica na prática:

Planejamento deixa de ser um evento único no início do projeto e passa a acontecer em dois níveis: um planejamento macro inicial (roadmap do produto) e um planejamento detalhado a cada sprint, onde apenas as próximas 2 semanas são planejadas com precisão. Isso reflete a realidade de que os requisitos mudam e planejar tudo com meses de antecedência frequentemente resulta em trabalho desperdiçado.

Análise de requisitos passa a ser contínua. O Product Backlog é um documento vivo, constantemente refinado. Novos requisitos descobertos ao longo do projeto entram no backlog e são priorizados, em vez de gerarem um processo burocrático de "mudança de escopo".

Design e desenvolvimento ocorrem dentro de cada sprint, focados nas funcionalidades daquela iteração. O XP adiciona práticas como refatoração contínua e programação em par para garantir que o design do código evolua com qualidade ao longo do tempo.

Testes no modelo ágil não são uma fase que acontece no final — são contínuos. Com testes automatizados e integração contínua, cada linha de código é testada imediatamente após ser escrita.

Entrega acontece ao final de cada sprint, não apenas ao final do projeto. Isso significa que o cliente começa a receber valor muito antes da conclusão total e pode dar feedback sobre funcionalidades reais, orientando o desenvolvimento das próximas iterações.

Manutenção se confunde com o próprio desenvolvimento — melhorias e correções identificadas pelos usuários entram no backlog e são priorizadas como qualquer outra funcionalidade, tratadas com o mesmo processo iterativo do ciclo principal.

O resultado é um ciclo de vida muito mais resiliente a mudanças, mais alinhado com as necessidades reais do cliente e que reduz drasticamente o risco de entregar um produto completo que não atende às expectativas — porque o cliente participa ativamente do processo desde a primeira sprint.